

The Embedded Rust Book

(Ukrainian)

Contents

1.	Вступ	5
1.0.1.	Для кого Вбудований Rust	5
1.0.2.	Область застосування	5
1.0.3.	Для кого Ця Книга	5
1.0.4.	Повторне використання цього матеріалу	6
1.1.	Зустрічайте залізо	7
1.1.1.	STM32F3DISCOVERY (скорочено “F3”)	7
1.2.	не перекладено	8
1.2.1.	не перекладено	8
1.2.2.	не перекладено	8
1.2.3.	не перекладено	8
1.2.4.	не перекладено	8
1.3.	не перекладено	8
1.3.1.	не перекладено	9
1.3.2.	cargo-binutils	9
1.3.3.	qemu-system-arm	9
1.4.	не перекладено	9
1.4.1.	не перекладено	9
1.4.2.	не перекладено	9
1.4.3.	не перекладено	9
1.4.4.	не перекладено	9
1.5.	не перекладено	10
1.5.1.	Linux	11
1.5.2.	macOS	12
1.5.3.	Windows	13
1.5.4.	не перекладено	13
2.	не перекладено	15
2.1.	QEMU	15
2.1.1.	не перекладено	15
2.1.2.	не перекладено	16
2.1.3.	не перекладено	16
2.1.4.	не перекладено	17
2.1.5.	не перекладено	19
2.1.6.	не перекладено	21
3.	не перекладено	23
3.0.1.	не перекладено	23
3.0.2.	не перекладено	23
3.0.3.	не перекладено	24
3.1.	не перекладено	27
3.1.1.	не перекладено	27
3.1.2.	не перекладено	27
3.1.3.	не перекладено	28
3.1.4.	не перекладено	29
3.2.	не перекладено	30
3.3.	не перекладено	32
3.3.1.	не перекладено	33
3.4.	не перекладено	33

3.4.1.	не перекладено	34
3.4.2.	не перекладено	36
3.4.3.	не перекладено	36
3.5.	не перекладено	37
4.	не перекладено	39
4.1.	не перекладено	39
4.2.	не перекладено	39
4.3.	не перекладено	39
4.4.	не перекладено	40
4.4.1.	не перекладено	40
4.4.2.	не перекладено	40
4.4.3.	не перекладено	40
4.4.4.	не перекладено	41
4.5.	не перекладено	42
4.5.1.	не перекладено	42
4.5.2.	не перекладено	42
4.6.	не перекладено	42
4.6.1.	не перекладено	42
4.6.2.	не перекладено	43
4.6.3.	не перекладено	43
4.6.4.	не перекладено	44
4.6.5.	не перекладено	44
5.	не перекладено	46
5.1.	не перекладено	46
5.1.1.	не перекладено	47
5.2.	не перекладено	47
5.2.1.	не перекладено	47
5.3.	не перекладено	48
5.3.1.	не перекладено	51
5.3.2.	не перекладено	54
5.4.	не перекладено	54
5.4.1.	не перекладено	54
5.4.2.	не перекладено	54
6.	не перекладено	55
6.0.1.	не перекладено	55
6.0.2.	не перекладено	55
7.	не перекладено	57
7.0.1.	не перекладено	57
7.0.2.	не перекладено	57
7.0.3.	не перекладено	58
7.0.4.	не перекладено	58
7.0.5.	не перекладено	59
7.0.6.	не перекладено	60
7.0.7.	не перекладено	61
7.0.8.	RTIC	63
7.0.9.	Embassy	63
7.0.10.	не перекладено	64
7.0.11.	не перекладено	64
8.	не перекладено	65

8.0.1.	не перекладено	65
8.0.2.	не перекладено	67
8.0.3.	не перекладено	67
9.	не перекладено	69
9.1.	не перекладено	69
9.1.1.	не перекладено	69
9.1.2.	не перекладено	69
9.1.3.	не перекладено	69
9.1.4.	не перекладено	69
9.1.5.	не перекладено	70
10.	не перекладено	74
10.0.1.	не перекладено	74
10.0.2.	не перекладено	75
10.0.3.	не перекладено	75
10.0.4.	не перекладено	75
10.0.5.	не перекладено	75
10.0.6.	не перекладено	76
10.0.7.	не перекладено	78
11.	не перекладено	79
11.0.1.	не перекладено	79
11.0.2.	не перекладено	79
11.1.	не перекладено	79
11.1.1.	не перекладено	79
11.1.2.	не перекладено	80
11.2.	не перекладено	81
11.2.1.	не перекладено	81
11.2.2.	не перекладено	81
11.2.3.	не перекладено	82
12.	не перекладено	83
12.1.	не перекладено	83
12.1.1.	не перекладено	83
12.1.2.	не перекладено	84
12.1.3.	не перекладено	84
12.2.	не перекладено	84
A.	не перекладено	87

1. Вступ

Ласкаво просимо до The Embedded Rust Book: ознайомчої книги про використання мови програмування Rust у вбудованих “Bare Metal” системах, таких як мікроконтролери.

1.0.1. Для кого Вбудований Rust

Вбудований Rust для кожного, хто бажає програмувати вбудовані системи використовуючи переваги концепцій вищого рівня та гарантій безпеки, які надає мова Rust. (Дивіться також [Для кого Rust](#))

1.0.2. Область застосування

Цілі цієї книги:

- Надати розробникам швидкий доступ до розробки вбудованих систем на Rust. Тобто як налаштувати середовище розробки.
- Поділитися найкращими *сучасними* практиками використання Rust для розробки вбудованих систем. Тобто як найкраще використовувати можливості мови Rust для написання більш коректних вбудованих програм.
- Слугувати кулінарною книгою у деяких випадках. Наприклад як поєднати C та Rust в одному проекті?

Ця книга намагається бути якомога більш загальною, але щоб полегшити роботу як читачам, так і авторам, у ній використовується архітектура ARM Cortex-M у всіх її прикладах. Однак книга не передбачає, що читач знайомий із цією конкретною архітектурою, і пояснює деталі, пов'язані з цією архітектурою, де потрібно.

1.0.3. Для кого Ця Книга

Ця книга розрахована на людей, які мають досвід роботи з вбудованими програмами або з Rust, але ми віримо, що всі, хто цікавиться програмуванням у вбудованому Rust, можуть щось отримати з цієї книги. Для тих, хто не має попередніх знань, ми пропонуємо прочитати розділ «Припущення та передумови» та надолужити пропущені знання, щоб отримати більше від книги та покращити свій досвід читання. Ви можете переглянути розділ «Інші ресурси», щоб знайти ресурси на теми, які, можливо, захочете надолужити.

1.0.3.1. Припущення та передумови

- Вам зручно користуватися мовою програмування Rust і ви вже писали, запускали та налагоджували програми Rust у середовищі робочого столу. Ви також повинні бути знайомі з ідіомами [2018 Edition](#), оскільки ця книга спрямована на Rust 2018.
- Вам зручно розробляти та налагоджувати вбудовані системи іншою мовою, наприклад C, C++ або Ada, і ви знайомі з такими поняттями, як:
 - Крос-компіляція
 - Периферійні пристрої з відображенням у пам'яті
 - Переривання
 - Загальні інтерфейси, такі як I2C, SPI, Serial тощо.

1.0.3.2. Інші ресурси

Якщо ви не знайомі з чимось із згаданого вище або вам потрібна додаткова інформація про конкретну тему, згадану в цій книзі, деякі з цих ресурсів можуть стати в нагоді.

не перекладено	не перекладено	не перекладено
Rust	Rust Book	не перекладено
Rust, Embedded	Discovery Book	не перекладено
Rust, Embedded	Embedded Rust Book-shelf	не перекладено
Rust, Embedded	Embedonomicon	не перекладено
Rust, Embedded	embedded FAQ	не перекладено
Rust, Embedded	Comprehensive Rust : Bare Metal	не перекладено
Interrupts	Interrupt	-
Memory-mapped IO/Peripherals	Memory-mapped I/O	-
SPI, UART, RS232, USB, I2C, TTL	Stack Exchange about SPI, UART, and other interfaces	-

1.0.3.3. Переклади

Цю книгу перекладено волонтерами. Якщо ви хочете, щоб ваш переклад був тут, будь ласка, відкрийте запит на злиття, щоб додати його.

- [Японською \(репозиторій\)](#)
- [Китайською \(репозиторій\)](#)

1.0.3.4. Як читати цю книгу

Ця книга зазвичай передбачає, що ви читаете її послідовно. Подальші розділи ґрунтуються на концепціях попередніх розділів, і попередні розділи можуть не заглиблюватися в деталі теми, повертаючись до неї в наступному розділі.

Для більшості наведених у цій книзі прикладів використовуватиметься розробна плата [STM32F3DISCOVERY](#) від STMicroelectronics. Ця плата заснована на архітектурі ARM Cortex-M, і хоча основні функції однакові для більшості процесорів на основі цієї архітектури, периферійні пристрої та інші деталі реалізації відрізняються у мікроконтролерах різних постачальників, і часто навіть різняться між сімействами мікроконтролерів одного й того ж постачальника.

З цієї причини ми пропонуємо придбати плату розробки [STM32F3DISCOVERY](#) щоб повторювати приклади з цієї книги.

1.0.3.5. Внесок у цю книгу

Робота над цією книгою координується в [цьому сховищі](#) і в основному розробляється [командою ресурсів](#).

Якщо у вас виникли проблеми з дотриманням інструкцій з цієї книги або ви вважаєте, що якийсь розділ книги недостатньо зрозумілий або складний для виконання, то це помилка, і про неї слід повідомити в [трекері випуску](#) цієї книги.

Запити на виправлення помилок і додавання нового матеріалу дуже вітаються!

1.0.4. Повторне використання цього матеріалу

Ця книга розповсюджується за наступними ліцензіями:

- Зразки коду та окремі проекти Cargo, що містяться в цій книзі, ліцензовано відповідно до умов [MIT License](#) та [Apache License v2.0](#).

- [Магнітометр](#) у складі мікросхеми [LSM303DLHC](#).
- [Гіроскоп](#) у складі мікросхеми [L3GD20](#).
- 8 користувачьких світлодіодів, розташованих у формі компаса.
- Другий мікроконтролер: [STM32F103](#). Цей мікроконтролер фактично є частиною програматора/відладчика, розміщеного на платі, і підключений до USB-порту з назвою “USB ST-LINK”.

Більш детальний перелік функцій та специфікацій плати можна знайти на сайті [STMicroelectronics](#).

Застереження: будьте обережні, подаючи на плату зовнішні сигнали. Виводи мікроконтролера STM32F303VCT6 мають номінальну напругу 3,3 вольтів. Для отримання додаткової інформації зверніться до розділу [6.2 Absolute maximum ratings section y Reference Manual](#)

1.2. не перекладено

не перекладено

не перекладено

1.2.1. не перекладено

не перекладено

1.2.2. не перекладено

не перекладено

1.2.2.1. не перекладено

не перекладено

1.2.3. не перекладено

не перекладено

1.2.3.1. не перекладено

не перекладено	no_std	std
не перекладено	*	✓
не перекладено	**	✓
не перекладено	✗	✓
не перекладено	✗	✓
не перекладено	✗	✓
не перекладено	✓	✓
не перекладено	✓	✗

не перекладено

не перекладено

не перекладено

1.2.4. не перекладено

- [RFC-1184](#)

1.3. не перекладено

не перекладено

не перекладено

не перекладено

1.3.1. не перекладено

не перекладено

не перекладено

1.3.2. cargo-binutils

не перекладено

не перекладено

1.3.3. qemu-system-arm

не перекладено

1.4. не перекладено

1.4.1. не перекладено

не перекладено

1.4.2. не перекладено

1.4.2.1. Probe-rs

не перекладено

1.4.2.2. OpenOCD (Open On-Chip Debugger)

не перекладено

1.4.3. не перекладено

не перекладено

не перекладено

1.4.3.1. Probe-rs Visual Studio Code Extension

не перекладено

1.4.3.2. TRACE32

не перекладено

1.4.3.3. GDB (GNU Debugger)

не перекладено

1.4.4. не перекладено

не перекладено

1.4.4.1. Rusty-probe

не перекладено

1.4.4.2. ST-Link

не перекладено

1.4.4.3. J-Link

не перекладено

1.4.4.4. MCU-Link

не перекладено

1.5. не перекладено

не перекладено

1.5.0.1. не перекладено

не перекладено

не перекладено

```
$ rustc -V
rustc 1.31.1 (b6c32da9b 2018-12-18)
```

Plain Text

не перекладено

не перекладено

```
rustup target add thumbv6m-none-eabi
```

Shell

не перекладено

```
rustup target add thumbv7m-none-eabi
```

Shell

не перекладено

```
rustup target add thumbv7em-none-eabi
```

Shell

не перекладено

```
rustup target add thumbv7em-none-eabihf
```

Shell

не перекладено

```
rustup target add thumbv8m.base-none-eabi
```

Shell

не перекладено

```
rustup target add thumbv8m.main-none-eabi
```

Shell

не перекладено

```
rustup target add thumbv8m.main-none-eabihf
```

Shell

1.5.0.2. cargo-binutils

```
cargo install cargo-binutils
```

Plain Text

```
rustup component add llvm-tools
```

не перекладено

1.5.0.3. cargo-generate

не перекладено

```
cargo install cargo-generate
```

Shell

не перекладено

1.5.0.4. не перекладено

не перекладено

- [Linux](#)
- [Windows](#)
- [macOS](#)

1.5.1. Linux

не перекладено

1.5.1.1. не перекладено

не перекладено

не перекладено

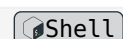
```
sudo apt install gdb-multiarch openocd qemu-system-arm
```



- Ubuntu 14.04 and 16.04

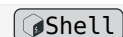
не перекладено

```
sudo apt install gdb-arm-none-eabi openocd qemu-system-arm
```



не перекладено

```
sudo dnf install gdb openocd qemu-system-arm
```



- Arch Linux

не перекладено

```
sudo pacman -S arm-none-eabi-gdb qemu-system-arm openocd
```



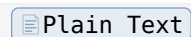
1.5.1.2. не перекладено

не перекладено

не перекладено

```
# STM32F3DISCOVERY rev A/B - ST-LINK/V2
ATTRS{idVendor}=="0483", ATTRS{idProduct}=="3748", TAG+="uaccess"

# STM32F3DISCOVERY rev C+ - ST-LINK/V2-1
ATTRS{idVendor}=="0483", ATTRS{idProduct}=="374b", TAG+="uaccess"
```



не перекладено

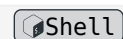
```
sudo udevadm control --reload-rules
```



не перекладено

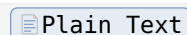
не перекладено

```
lsusb
```

 Shell

не перекладено

```
(..)
```

 Plain Text

```
Bus 001 Device 018: ID 0483:374b STMicroelectronics ST-LINK/V2.1
```

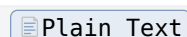
```
(..)
```

не перекладено

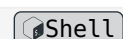
```
ls -l /dev/bus/usb/001/018
```

 Shell

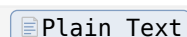
```
crw-----+ 1 root root 189, 17 Sep 13 12:34 /dev/bus/usb/001/018
```

 Plain Text

```
getfacl /dev/bus/usb/001/018 | grep user
```

 Shell

```
user::rw-
```

 Plain Text

```
user:you:rw-
```

не перекладено

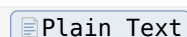
не перекладено

1.5.2. macOS

не перекладено

1.5.2.1. не перекладено

```
$ # GDB
```

 Plain Text

```
$ brew install arm-none-eabi-gdb
```

```
$ # OpenOCD
```

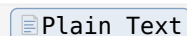
```
$ brew install openocd
```

```
$ # QEMU
```

```
$ brew install qemu
```

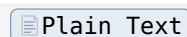
не перекладено

```
$ brew install --HEAD openocd
```

 Plain Text

1.5.2.2. не перекладено

```
$ # GDB
```

 Plain Text

```
$ sudo port install arm-none-eabi-gcc
```

```
$ # OpenOCD
```

```
$ sudo port install openocd
```

```
$ # QEMU
```

```
$ sudo port install qemu
```

не перекладено

1.5.3. Windows

1.5.3.1. arm-none-eabi-gdb

не перекладено

```
$ arm-none-eabi-gdb -v
GNU gdb (GNU Tools for Arm Embedded Processors 7-2018-q2-update) 8.1.0.20180315-git
(..)
```

Plain Text

1.5.3.2. OpenOCD

не перекладено

не перекладено

```
$ openocd -v
Open On-Chip Debugger 0.10.0
(..)
```

Plain Text

1.5.3.3. QEMU

не перекладено

1.5.3.4. не перекладено

не перекладено

не перекладено

1.5.4. не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

```
openocd -f interface/stlink.cfg -f target/stm32f3x.cfg
```

Shell

не перекладено

не перекладено

```
Open On-Chip Debugger 0.10.0
Licensed under GNU GPL v2
For bug reports, read
    http://openocd.org/doc/doxygen/bugs.html
Info : auto-selecting first available session transport "hla_swd". To override use
'transport select <transport>'.
adapter speed: 1000 kHz
adapter_nsrst_delay: 100
Info : The selected transport took over low-level target control. The results might
differ compared to plain JTAG/SWD
```

Plain Text

```
none separate
Info : Unable to match requested speed 1000 kHz, using 950 kHz
Info : Unable to match requested speed 1000 kHz, using 950 kHz
Info : clock speed 950 kHz
Info : STLINK v2 JTAG v27 API v2 SWIM v15 VID 0x0483 PID 0x374B
Info : using stlink api v2
Info : Target voltage: 2.919881
Info : stm32f3x.cpu: hardware has 6 breakpoints, 4 watchpoints
```

не перекладено

не перекладено

```
openocd -f interface/stlink-v2.cfg -f target/stm32f3x.cfg
```

 Shell

```
openocd -f interface/stlink-v2-1.cfg -f target/stm32f3x.cfg
```

 Shell

не перекладено

не перекладено

не перекладено

2. не перекладено

не перекладено

2.1. QEMU

не перекладено

не перекладено

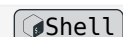
2.1.1. не перекладено

не перекладено

2.1.1.1. не перекладено

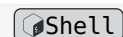
не перекладено

```
cargo install cargo-generate
```

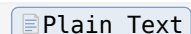


не перекладено

```
cargo generate --git https://github.com/knurling-rs/app-template
```



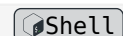
```
Project Name: app
```



```
Creating project called `app`...
```

```
Done! New project created /tmp/app
```

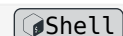
```
cd app
```



2.1.1.2. не перекладено

не перекладено

```
git clone https://github.com/rust-embedded/cortex-m-quickstart app
```



```
cd app
```

не перекладено

```
[package]
authors = ["{{authors}}"] # "{{authors}}" -> "John Smith"
edition = "2018"
name = "{{project-name}}" # "{{project-name}}" -> "app"
version = "0.1.0"

# ..

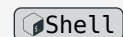
[[bin]]
name = "{{project-name}}" # "{{project-name}}" -> "app"
test = false
bench = false
```



2.1.1.3. не перекладено

не перекладено

```
curl -LO https://github.com/rust-embedded/cortex-m-quickstart/archive/master.zip
```



```
unzip master.zip
mv cortex-m-quickstart-master app
cd app
```

не перекладено

не перекладено

2.1.2. не перекладено

не перекладено

```
#![no_std]
#![no_main]

use panic_halt as _;

use cortex_m_rt::entry;

#[entry]
fn main() -> ! {
    loop {
        // не перекладено
    }
}
```

 Rust

не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

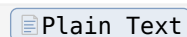
2.1.3. не перекладено

не перекладено

```
MEMORY
{
    /* не перекладено
    FLASH : ORIGIN = 0x00000000, LENGTH = 256K
    RAM : ORIGIN = 0x20000000, LENGTH = 64K
    */

    /* не перекладено
    /* _stack_start = ORIGIN(RAM) + LENGTH(RAM); */

    /* не перекладено
    /* _stext = ORIGIN(FLASH) + 0x400; */
```

 Plain Text


```
/* не перекладено
/* SECTIONS {
    .ram2bss (NOLOAD) : ALIGN(4) {
        *(.ram2bss);
        . = ALIGN(4);
    } > RAM2
    } INSERT AFTER .bss;
*/
```

не перекладено

```
tail -n6 .cargo/config.toml
```

```
[build]
# не перекладено
# target = "thumbv6m-none-eabi" # Cortex-M0 and Cortex-M0+
target = "thumbv7m-none-eabi" # Cortex-M3
# target = "thumbv7em-none-eabi" # Cortex-M4 and Cortex-M7 (no FPU)
# target = "thumbv7em-none-eabihf" # Cortex-M4F and Cortex-M7F (with FPU)
```

не перекладено

```
rustup target add thumbv7m-none-eabi
```

не перекладено

```
cargo build --target thumbv7m-none-eabi
cargo build
```

2.1.4. не перекладено

не перекладено

не перекладено

```
cargo readobj --bin app -- --file-headers
```

не перекладено

ELF Header:

```
Magic:  7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00
Class:                                     ELF32
Data:                                       2's complement, little endian
Version:                                   1 (current)
OS/ABI:                                    UNIX - System V
ABI Version:                               0x0
Type:                                       EXEC (Executable file)
Machine:                                   ARM
Version:                                   0x1
Entry point address:                       0x405
```

Start of program headers:	52 (bytes into file)
Start of section headers:	153204 (bytes into file)
Flags:	0x5000200
Size of this header:	52 (bytes)
Size of program headers:	32 (bytes)
Number of program headers:	2
Size of section headers:	40 (bytes)
Number of section headers:	19
Section header string table index:	18

не перекладено

```
cargo size --bin app --release -- -A
```



не перекладено

app :

Plain Text

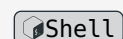
section	size	addr
.vector_table	1024	0x0
.text	92	0x400
.rodata	0	0x45c
.data	0	0x20000000
.bss	0	0x20000000
.debug_str	2958	0x0
.debug_loc	19	0x0
.debug_abbrev	567	0x0
.debug_info	4929	0x0
.debug_ranges	40	0x0
.debug_machinfo	1	0x0
.debug_pubnames	2035	0x0
.debug_pubtypes	1892	0x0
.ARM.attributes	46	0x0
.debug_frame	100	0x0
.debug_line	867	0x0
Total	14570	

не перекладено

не перекладено

не перекладено

```
cargo objdump --bin app --release -- --disassemble --no-show-raw-insn --
print-imm-hex
```



не перекладено

не перекладено

app: file format ELF32-arm-little

Plain Text

Disassembly of section .text:

main:

400: bl #0x256

404: b #-0x4 <main+0x4>

Reset:

406: bl #0x24e

40a: movw r0, #0x0

< .. truncated any more instructions .. >

DefaultHandler_:

656: b #-0x4 <DefaultHandler_>

UsageFault:

657: strb r7, [r4, #0x3]

DefaultPreInit:

658: bx lr

__pre_init:

659: strb r7, [r0, #0x1]

__nop:

65a: bx lr

HardFaultTrampoline:

65c: mrs r0, msp

660: b #-0x2 <HardFault_>

HardFault_:

662: b #-0x4 <HardFault_>

HardFault:

663: <unknown>

2.1.5. не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

cargo remove defmt-rtt

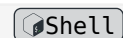
Shell

```
cargo add defmt-semihosting
```

не перекладено

не перекладено

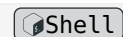
```
cargo build --bin hello
```



не перекладено

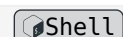
не перекладено

```
qemu-system-arm \
  -cpu cortex-m3 \
  -machine lm3s6965evb \
  -nographic \
  -semihosting-config enable=on,target=native \
  -kernel target/thumbv7m-none-eabi/debug/hello
```

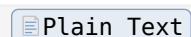


не перекладено

```
git clone git@github.com:knurling-rs/defmt.git
cd defmt/qemu-run/
cargo run -- --machine lm3s6965evb ../qemu-rs/target/thumbv7m-none-eabi/debug/hello
```



Hello, world!

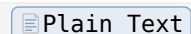


не перекладено

```
echo $?
```



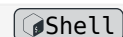
```
0
```



не перекладено

не перекладено

```
head -n3 .cargo/config.toml
```

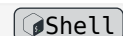


```
[target.thumbv7m-none-eabi]
# не перекладено
runner = "qemu-system-arm -cpu cortex-m3 -machine lm3s6965evb -nographic -
semihosting-config enable=on,target=native -kernel"
```

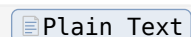


не перекладено

```
cargo ru(
  level: 2 + whole
)--release
```



```
Compiling app v0.1.0 (file:///tmp/app)
Finished release [optimized + debuginfo] target(s) in 0.26s
```



```
Running `qemu-system-arm -cpu cortex-m3 -machine lm3s6965evb -nographic -
semihosting-config enable=on,target=native -kernel target/thumbv7m-none-eabi/
release/examples/hello`
```

Hello, world!

2.1.6. не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

```
qemu-system-arm \
  -cpu cortex-m3 \
  -machine lm3s6965evb \
  -nographic \
  -semihosting-config enable=on,target=native \
  -gdb tcp::3333 \
  -S \
  -kernel target/thumbv7m-none-eabi/debug/examples/hello
```

Shell

не перекладено

не перекладено

```
gdb-multiarch -q target/thumbv7m-none-eabi/debug/examples/hello
```

Shell

не перекладено

не перекладено

```
target remote :3333
```

Shell

```
Remote debugging using :3333
```

Plain Text

```
Reset () at $REGISTRY/cortex-m-rt-0.6.1/src/lib.rs:473
```

```
473      pub unsafe extern "C" fn Reset() -> ! {
```

не перекладено

не перекладено

```
core::num::bignum::Big32x40::mul_small () at src/libcore/num/bignum.rs:254 src/
libcore/num/bignum.rs: No such file or directory.
```

не перекладено

не перекладено

```
list main
```

Shell

не перекладено

```

6      use panic_halt as _;
7
8      use cortex_m_rt::entry;
9      use cortex_m_semihosting::{debug, hprintln};
10
11     #[entry]
12     fn main() -> ! {
13         hprintln!("Hello, world!").unwrap();
14
15         // exit QEMU

```

 Plain Text

не перекладено

```
break 13
```

 Shell

не перекладено

```
continue
```

 Shell

```
Continuing.
```

 Plain Text

```

Breakpoint 1, hello::__cortex_m_rt_main () at examples\hello.rs:13
13      hprintln!("Hello, world!").unwrap();

```

не перекладено

```
next
```

 Shell

```
16      debug::exit(debug::EXIT_SUCCESS);
```

 Plain Text

не перекладено

```

$ qemu-system-arm (..)
Hello, world!

```

 Plain Text

не перекладено

```
next
```

 Shell

```
[Inferior 1 (Remote target) exited normally]
```

 Plain Text

не перекладено

```
quit
```

 Shell

3. не перекладено

не перекладено

3.0.1. не перекладено

не перекладено

не перекладено

не перекладено

3.0.2. не перекладено

не перекладено

```
$ cargo generate --git https://github.com/rust-embedded/cortex-m-quickstart
```

Plain Text

```
Project Name: app
```

```
Creating project called `app`...
```

```
Done! New project created /tmp/app
```

```
$ cd app
```

не перекладено

```
tail -n5 .cargo/config.toml
```

Shell

```
# не перекладено
```

TOML

```
# target = "thumbv6m-none-eabi" # Cortex-M0 and Cortex-M0+
```

```
# target = "thumbv7m-none-eabi" # Cortex-M3
```

```
# target = "thumbv7em-none-eabi" # Cortex-M4 and Cortex-M7 (no FPU)
```

```
target = "thumbv7em-none-eabihf" # Cortex-M4F and Cortex-M7F (with FPU)
```

не перекладено

не перекладено

не перекладено

```
$ cat memory.x
```

Plain Text

```
/* Linker script for the STM32F303VCT6 */
```

```
MEMORY
```

```
{
```

```
/* NOTE 1 K = 1 KiBi = 1024 bytes */
```

```
FLASH : ORIGIN = 0x08000000, LENGTH = 256K
```

```
RAM : ORIGIN = 0x20000000, LENGTH = 40K
```

```
}
```

не перекладено

не перекладено

не перекладено

```
#[entry]
fn main() -> ! {
    hprintln!("Hello, world!").unwrap();

    // не перекладено

    loop {}
}
```

не перекладено

```
cargo build --example hello
```

3.0.3. не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

```
cat openocd.cfg
```

```
# не перекладено
source [find interface/stlink.cfg]

# не перекладено
# source [find interface/stlink-v2.cfg]

source [find target/stm32f3x.cfg]
```

не перекладено

```
$ openocd
Open On-Chip Debugger 0.10.0
Licensed under GNU GPL v2
For bug reports, read
    http://openocd.org/doc/doxygen/bugs.html
Info : auto-selecting first available session transport "hla_swd". To override use
'transport select <transport>'.
adapter speed: 1000 kHz
adapter_nsrst_delay: 100
Info : The selected transport took over low-level target control. The results might
differ compared to plain JTAG/SWD
none separate
Info : Unable to match requested speed 1000 kHz, using 950 kHz
Info : Unable to match requested speed 1000 kHz, using 950 kHz
Info : clock speed 950 kHz
```



```
Info : STLINK v2 JTAG v27 API v2 SWIM v15 VID 0x0483 PID 0x374B
Info : using stlink api v2
Info : Target voltage: 2.913879
Info : stm32f3x.cpu: hardware has 6 breakpoints, 4 watchpoints
```

не перекладено

```
gdb-multiarch -q target/thumbv7em-none-eabihf/debug/examples/hello
```

Plain Text

не перекладено

не перекладено

```
(gdb) target remote :3333
Remote debugging using :3333
0x00000000 in ?? ()
```

Shell

не перекладено

```
(gdb) load
Loading section .vector_table, size 0x400 lma 0x8000000
Loading section .text, size 0x1518 lma 0x8000400
Loading section .rodata, size 0x414 lma 0x8001918
Start address 0x08000400, load size 7468
Transfer rate: 13 KB/sec, 2489 bytes/write.
```

Shell

не перекладено

```
(gdb) monitor arm semihosting enable
semihosting is enabled
```

Shell

не перекладено

не перекладено

```
(gdb) break main
Breakpoint 1 at 0x8000490: file examples/hello.rs, line 11.
Note: automatically using hardware breakpoints for read-only addresses.
```

Shell

```
(gdb) continue
Continuing.
```

```
Breakpoint 1, hello::__cortex_m_rt_main_trampoline () at examples/hello.rs:11
11      #[entry]
```

не перекладено

не перекладено

```
(gdb) step
```

Shell

```
halted: PC: 0x08000496
hello::__cortex_m_rt_main () at examples/hello.rs:13
13          hprintln!("Hello, world!").unwrap();
```

не перекладено

```
$ openocd
(..)
Info : halted: PC: 0x08000502
Hello, world!
Info : halted: PC: 0x080004ac
Info : halted: PC: 0x080004ae
Info : halted: PC: 0x080004b0
Info : halted: PC: 0x080004b4
Info : halted: PC: 0x080004b8
Info : halted: PC: 0x080004bc
```

Shell

не перекладено

не перекладено

```
(gdb) quit
A debugging session is active.

        Inferior 1 [Remote target] will be detached.

Quit anyway? (y or n)
```

Shell

не перекладено

```
cat openocd.gdb
```

Shell

```
target extended-remote :3333
```

Plain Text

```
# print demangled symbols
set print asm-demangle on
```

```
# detect unhandled exceptions, hard faults and panics
break DefaultHandler
break HardFault
break rust_begin_unwind
```

```
monitor arm semihosting enable
```

```
load
```

```
# start the process but immediately halt the processor
stepi
```

не перекладено

не перекладено

```
head -n10 .cargo/config.toml
```

```
[target.thumbv7m-none-eabi]
# не перекладено
# runner = "qemu-system-arm -cpu cortex-m3 -machine lm3s6965evb -nographic -
semihosting-config enable=on,target=native -kernel"

[target.'cfg(all(target_arch = "arm", target_os = "none"))']
# не перекладено
runner = "arm-none-eabi-gdb -x openocd.gdb"
# runner = "gdb-multiarch -x openocd.gdb"
# runner = "gdb -x openocd.gdb"
```

```
$ cargo run --example hello
(..)
Loading section .vector_table, size 0x400 lma 0x8000000
Loading section .text, size 0x1e70 lma 0x8000400
Loading section .rodata, size 0x61c lma 0x8002270
Start address 0x800144e, load size 10380
Transfer rate: 17 KB/sec, 3460 bytes/write.
(gdb)
```

3.1. не перекладено

не перекладено

не перекладено

3.1.1. не перекладено

не перекладено

не перекладено

не перекладено

3.1.2. не перекладено

не перекладено

```
#![no_std]
#![no_main]
use cortex_m::peripheral::{syst, Peripherals};
use cortex_m_rt::entry;
use panic_halt as _;

#[entry]
fn main() -> ! {
    let peripherals = Peripherals::take().unwrap();
    let mut systick = peripherals.SYST;
    systick.set_clock_source(syst::SystClkSource::Core);
    systick.set_reload(1_000);
```

```

systick.clear_current();
systick.enable_counter();
while !systick.has_wrapped() {
    // не перекладено
}

loop {}
}

```

не перекладено

3.1.3. не перекладено

не перекладено

```

#![no_std]
#![no_main]

use panic_halt as _; // не перекладено

use cortex_m_rt::entry;
use tm4c123x;

#[entry]
pub fn init() -> (Delay, Leds) {
    let cp = cortex_m::Peripherals::take().unwrap();
    let p = tm4c123x::Peripherals::take().unwrap();

    let pwm = p.PWM0;
    pwm.ctl.write(|w| w.globalsync0().clear_bit());
    // не перекладено
    pwm._2_ctl.write(|w| w.enable().set_bit().mode().set_bit());
    pwm._2_gena.write(|w| w.actcmpau().zero().actcmpad().one());
    // не перекладено
    pwm._2_load.write(|w| unsafe { w.load().bits(263) });
    pwm._2_cmpa.write(|w| unsafe { w.compa().bits(64) });
    pwm.enable.write(|w| w.pwm4en().set_bit());
}

```

 Rust

не перекладено

3.1.3.1. не перекладено

не перекладено

```

if pwm.ctl.read().globalsync0().is_set() {
    // не перекладено
}

```

 Rust

3.1.3.2. не перекладено

не перекладено

```
pwm0.ctl.write(|w| w.globalsync0().clear_bit());
```

 Rust

3.1.3.3. не перекладено

не перекладено

```
pwm0.ctl.modify(|r, w| w.globalsync0().clear_bit());
```

 Rust

не перекладено

```
uint32_t temp = pwm0.ctl.read();
temp |= PWM0_CTL_GLOBALSYNC0;
pwm0.ctl.write(temp);
uint32_t temp2 = pwm0.enable.read();
temp2 |= PWM0_ENABLE_PWM4EN;
pwm0.enable.write(temp); // не перекладено
```

 C

3.1.4. не перекладено

не перекладено

не перекладено

```
#![no_std]
#![no_main]

use panic_halt as _; // panic handler

use cortex_m_rt::entry;
use tm4c123x_hal as hal;
use tm4c123x_hal::prelude::*;
use tm4c123x_hal::serial::{NewlineMode, Serial};
use tm4c123x_hal::sysctl;

#[entry]
fn main() -> ! {
    let p = hal::Peripherals::take().unwrap();
    let cp = hal::CorePeripherals::take().unwrap();

    // не перекладено
    let mut sc = p.SYSCTL.constrain();
    // не перекладено
    sc.clock_setup.oscillator = sysctl::Oscillator::Main(
        sysctl::CrystalFrequency::_16mhz,
        sysctl::SystemClock::UsePll(sysctl::PllOutputFrequency::_80_00mhz),
    );
    // не перекладено
    let clocks = sc.clock_setup.freeze();

    // не перекладено
```

 Rust

```

let mut porta = p.GPIO_PORTA.split(&sc.power_control);

// не перекладено
let uart = Serial::uart0(
    p.UART0,
    // не перекладено
    porta
        .pa1
        .into_af_push_pull::<hal::gpio::AF1>(&mut porta.control),
    // не перекладено
    porta
        .pa0
        .into_af_push_pull::<hal::gpio::AF1>(&mut porta.control),
    // не перекладено
    (),
    (),
    // не перекладено
    115200_u32.bps(),
    // не перекладено
    NewlineMode::SwapLFtoCRLF,
    // не перекладено
    &clocks,
    // не перекладено
    &sc.power_control,
);

loop {
    writeln!(uart, "Hello, World!\r\n").unwrap();
}
}

```

3.2. не перекладено

не перекладено

не перекладено

```

#![no_main]
#![no_std]

use panic_halt as _;

use cortex_m_rt::entry;
use cortex_m_semihosting::hprintln;

#[entry]
fn main() -> ! {
    hprintln!("Hello, world!").unwrap();
}

```



```
loop {}
}
```

не перекладено

```
$ openocd
(..)
Hello, world!
(..)
```

Plain Text

не перекладено

```
(gdb) monitor arm semihosting enable
semihosting is enabled
```

Shell

не перекладено

```
$ # не перекладено
$ cargo run
    Running `qemu-system-arm (..)`
Hello, world!
```

Plain Text

не перекладено

```
#![no_main]
#![no_std]

use panic_halt as _;

use cortex_m_rt::entry;
use cortex_m_semihosting::debug;

#[entry]
fn main() -> ! {
    let roses = "blue";

    if roses == "red" {
        debug::exit(debug::EXIT_SUCCESS);
    } else {
        debug::exit(debug::EXIT_FAILURE);
    }

    loop {}
}
```

Rust

```
$ cargo run
    Running `qemu-system-arm (..)`
```

Plain Text

```
$ echo $?
1
```

не перекладено

не перекладено

```
#![no_main]
#![no_std]

use panic_semihosting as _; // features = ["exit"]

use cortex_m_rt::entry;
use cortex_m_semihosting::debug;

#[entry]
fn main() -> ! {
    let roses = "blue";

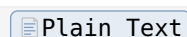
    assert_eq!(roses, "red");

    loop {}
}
```



```
$ cargo run
    Running `qemu-system-arm (...)
panicked at 'assertion failed: `(left == right)`
  left: `"blue"`,
 right: `"red"`', examples/hello.rs:15:5

$ echo $?
1
```



не перекладено

```
panic-semihosting = { version = "VERSION", features = ["exit"] }
```



не перекладено

3.3. не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

```
#![no_main]
```




```

#![no_std]

// не перекладено
#[cfg(debug_assertions)]
use panic_halt as _;

// не перекладено
#[cfg(not(debug_assertions))]
use panic_abort as _;

// ..

```

не перекладено

не перекладено

3.3.1. не перекладено

не перекладено

```

#![no_main]
#![no_std]

use panic_semihosting as _;

use cortex_m_rt::entry;

#[entry]
fn main() -> ! {
    let xs = [0, 1, 2];
    let i = xs.len();
    let _y = xs[i]; // не перекладено

    loop {}
}

```

 Rust

не перекладено

```

$ cargo run
Running `qemu-system-arm -cpu cortex-m3 -machine lm3s6965evb (...)
panicked at 'index out of bounds: the len is 3 but the index is 4', src/
main.rs:12:13

```

 Plain Text

не перекладено

3.4. не перекладено

не перекладено

не перекладено

```
// не перекладено
```



```
#[exception]
fn SysTick() {
    // ..
}
```

не перекладено

не перекладено

```
#[exception]
```



```
fn SysTick() {
    static mut COUNT: u32 = 0;
    // не перекладено
    *COUNT += 1;
}
```

не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

3.4.1. не перекладено

не перекладено

не перекладено

```
#![deny(unsafe_code)]
```



```
#![no_main]
```

```
#![no_std]
```

```
use panic_halt as _;
```

```
use core::fmt::Write;
```

```
use cortex_m::peripheral::syst::SystClkSource;
```

```
use cortex_m_rt::{entry, exception};
```

```
use cortex_m_semihosting::{
    debug,
    hio::{self, HostStream},
};
```

```
#[entry]
```

```
fn main() -> ! {
```

```
    let p = cortex_m::Peripherals::take().unwrap();
```

```

let mut syst = p.SYST;

// не перекладено
syst.set_clock_source(SystClkSource::Core);
// не перекладено
syst.set_reload(12_000_000);
syst.clear_current();
syst.enable_counter();
syst.enable_interrupt();

loop {}
}

#[exception]
fn SysTick() {
    static mut COUNT: u32 = 0;
    static mut STDOUT: Option<HostStream> = None;

    *COUNT += 1;

    // не перекладено
    if STDOUT.is_none() {
        *STDOUT = hio::hstdout().ok();
    }

    if let Some(hstdout) = STDOUT.as_mut() {
        write!(hstdout, "{}", *COUNT).ok();
    }

    // не перекладено
    if *COUNT == 9 {
        // не перекладено
        debug::exit(debug::EXIT_SUCCESS);
    }
}

```

```
tail -n5 Cargo.toml
```



```

[dependencies]
cortex-m = "0.5.7"
cortex-m-rt = "0.6.3"
panic-halt = "0.2.0"
cortex-m-semihosting = "0.3.1"

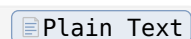
```



```

$ cargo run --release
    Running `qemu-system-arm -cpu cortex-m3 -machine lm3s6965evb (...)
123456789

```



не перекладено

3.4.2. не перекладено

не перекладено

```
fn DefaultHandler() {
    loop {}
}
```

 Rust

не перекладено

не перекладено

```
#[exception]
fn DefaultHandler(irqn: i16) {
    // не перекладено
}
```

 Rust

не перекладено

3.4.3. не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

```
#![no_main]
#![no_std]

use panic_halt as _;

use core::fmt::Write;
use core::ptr;

use cortex_m_rt::{entry, exception, ExceptionFrame};
use cortex_m_semihosting::hio;

#[entry]
fn main() -> ! {
    // не перекладено
    unsafe {
        ptr::read_volatile(0x3FFF_0000 as *const u32);
    }

    loop {}
}
```

 Rust

```
#[exception]
fn HardFault(ef: &ExceptionFrame) -> ! {
    if let Ok(mut hstdout) = hio::hstdout() {
        writeln!(hstdout, "{:#?}", ef).ok();
    }

    loop {}
}
```

не перекладено

```
$ openocd
(..)
ExceptionFrame {
    r0: 0x3fff0000,
    r1: 0x00000003,
    r2: 0x080032e8,
    r3: 0x00000000,
    r12: 0x00000000,
    lr: 0x080016df,
    pc: 0x080016e2,
    xpsr: 0x61000000,
}
```

не перекладено

не перекладено

```
$ cargo objdump --bin app --release -- -d --no-show-raw-insn --print-imm-hex
(..)
ResetTrampoline:
8000942:      movw    r0, #0xfffe
8000946:      movt    r0, #0x3fff
800094a:      ldr     r0, [r0]
800094c:      b       #-0x4 <ResetTrampoline+0xa>
```

не перекладено

3.5. не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

```
rust
use lm3s6965::interrupt; // не перекладено
```

```
// не перекладено
#[interrupt]
fn TIMER2A() {
    // ..
    // не перекладено
}
```

не перекладено

не перекладено

```
#[interrupt]
fn TIMER2A() {
    static mut COUNT: u32 = 0;

    // не перекладено
    *COUNT += 1;
}
```



не перекладено

4. не перекладено

4.1. не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

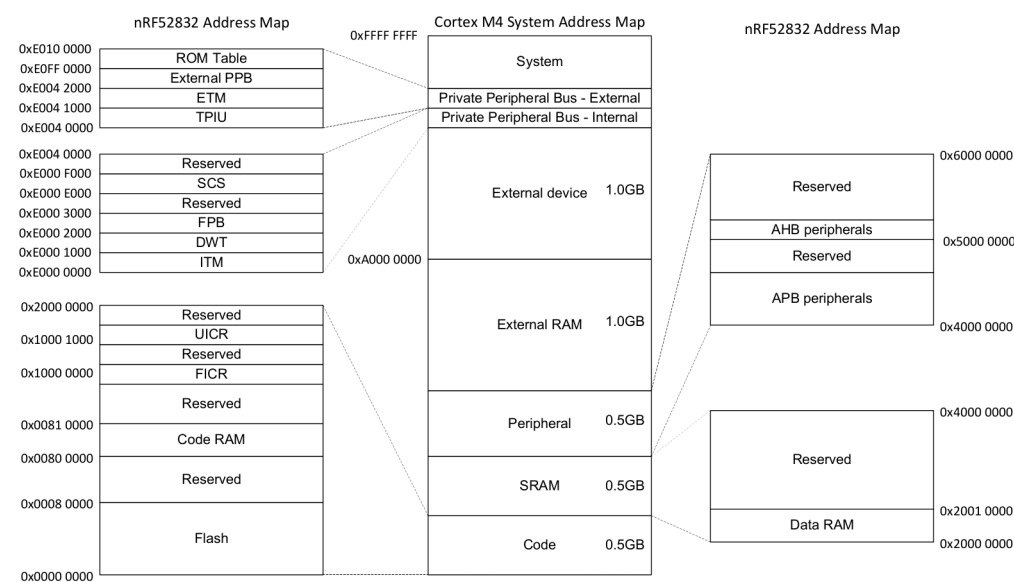
не перекладено

4.2. не перекладено

не перекладено

не перекладено

не перекладено



[Nordic nRF52832 Datasheet \(pdf\)](#)

4.3. не перекладено

не перекладено

не перекладено

31.6.8 FREQUENCY

Address offset: 0x524

SPI frequency

Bit number					31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Id					A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A		
Reset 0x04000000					0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
Id	RW	Field	Value	Id	Value	Description																																
A	RW	FREQUENCY			SPI master data rate																																	
			K125		0x02000000	125 kbps																																
			K250		0x04000000	250 kbps																																
			K500		0x08000000	500 kbps																																
			M1		0x10000000	1 Mbps																																
			M2		0x20000000	2 Mbps																																
			M4		0x40000000	4 Mbps																																
			M8		0x80000000	8 Mbps																																

[Nordic nRF52832 Datasheet \(pdf\)](#)

не перекладено

4.4. не перекладено

4.4.1. не перекладено

не перекладено

не перекладено	не перекладено	не перекладено	не перекладено
0x00	SYST_CSR	не перекладено	32 bits
0x04	SYST_RVR	не перекладено	32 bits
0x08	SYST_CVR	не перекладено	32 bits
0x0C	SYST_CALIB	не перекладено	32 bits

4.4.2. не перекладено

не перекладено

```
#[repr(C)]
struct SysTick {
    pub csr: u32,
    pub rvr: u32,
    pub cvr: u32,
    pub calib: u32,
}
```

не перекладено

```
let systick = 0xE000_E010 as *mut SysTick;
let time = unsafe { (*systick).cvr };
```

4.4.3. не перекладено

не перекладено

не перекладено

```
let systick = unsafe { &mut *(0xE000_E010 as *mut SysTick) };
let time = unsafe { core::ptr::read_volatile(&mut systick.cvr) };
```


не перекладено

```
use volatile_register::{RW, R0};

#[repr(C)]
struct SysTick {
    pub csr: RW<u32>,
    pub rvr: RW<u32>,
    pub cvr: RW<u32>,
    pub calib: R0<u32>,
}

fn get_systick() -> &'static mut SysTick {
    unsafe { &mut *(0xE000_E010 as *mut SysTick) }
}

fn get_time() -> u32 {
    let systick = get_systick();
    systick.cvr.read()
}
```

 Rust

не перекладено

4.4.4. не перекладено

не перекладено

не перекладено

```
use volatile_register::{RW, R0};

pub struct SystemTimer {
    p: &'static mut RegisterBlock
}

#[repr(C)]
struct RegisterBlock {
    pub csr: RW<u32>,
    pub rvr: RW<u32>,
    pub cvr: RW<u32>,
    pub calib: R0<u32>,
}

impl SystemTimer {
    pub fn new() -> SystemTimer {
        SystemTimer {
            p: unsafe { &mut *(0xE000_E010 as *mut RegisterBlock) }
        }
    }
}
```

 Rust

```

pub fn get_time(&self) -> u32 {
    self.p.cvr.read()
}

pub fn set_reload(&mut self, reload_value: u32) {
    unsafe { self.p.rvr.write(reload_value) }
}

pub fn example_usage() -> String {
    let mut st = SystemTimer::new();
    st.set_reload(0x00FF_FFFF);
    format!("Time is now 0x{:08x}", st.get_time())
}

```

не перекладено

```

fn thread1() {
    let mut st = SystemTimer::new();
    st.set_reload(2000);
}

fn thread2() {
    let mut st = SystemTimer::new();
    st.set_reload(1000);
}

```

 Rust

не перекладено

4.5. не перекладено

не перекладено

4.5.1. не перекладено

не перекладено

4.5.2. не перекладено

не перекладено

не перекладено

не перекладено

4.6. не перекладено

не перекладено

не перекладено

4.6.1. не перекладено

не перекладено

```
static mut THE_SERIAL_PORT: SerialPort = SerialPort;
```



```
fn main() {
    let _ = unsafe {
        THE_SERIAL_PORT.read_speed();
    };
}
```

не перекладено

4.6.2. не перекладено

не перекладено

```
struct Peripherals {
    serial: Option<SerialPort>,
}
```



```
impl Peripherals {
    fn take_serial(&mut self) -> SerialPort {
        let p = replace(&mut self.serial, None);
        p.unwrap()
    }
}

static mut PERIPHERALS: Peripherals = Peripherals {
    serial: Some(SerialPort),
};
```

не перекладено

```
fn main() {
    let serial_1 = unsafe { PERIPHERALS.take_serial() };
    // не перекладено
    // let serial_2 = unsafe { PERIPHERALS.take_serial() };
}
```



не перекладено

не перекладено

4.6.3. не перекладено

не перекладено

```
use cortex_m::singleton;
```



```
fn main() {
    // не перекладено
    let x: &'static mut bool =
        singleton!(bool = false).unwrap();
}
```

не перекладено

не перекладено

```
// cortex-m-rtic v0.5.x
#[rtic::app(device = lm3s6965, peripherals = true)]
const APP: () = {
    #[init]
    fn init(cx: init::Context) {
        static mut X: u32 = 0;

        // не перекладено
        let core: cortex_m::Peripherals = cx.core;

        // не перекладено
        let device: lm3s6965::Peripherals = cx.device;
    }
}
```

4.6.4. не перекладено

не перекладено

```
impl SerialPort {
    const SER_PORT_SPEED_REG: *mut u32 = 0x4000_1000 as _;

    fn read_speed(
        &self // не перекладено
    ) -> u32 {
        unsafe {
            ptr::read_volatile(Self::SER_PORT_SPEED_REG)
        }
    }
}
```

не перекладено

не перекладено

```
fn main() {
    // не перекладено
    // SerialPort::read_speed();

    let serial_1 = unsafe { PERIPHERALS.take_serial() };

    // не перекладено
    let _ = serial_1.read_speed();
}
```

4.6.5. не перекладено

не перекладено

не перекладено

```
fn setup_spi_port(  
    spi: &mut SpiPort,  
    cs_pin: &mut GpioPin  
) -> Result<()> {  
    // ...  
}
```



не перекладено

```
fn read_button(gpio: &GpioPin) -> bool {  
    // ...  
}
```



не перекладено

5. не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

5.1. не перекладено

не перекладено

```
pub mod foo_module {
    #[derive(Debug)]
    pub struct Foo {
        inner: u32,
    }

    pub struct FooBuilder {
        a: u32,
        b: u32,
    }

    impl FooBuilder {
        pub fn new(starter: u32) -> Self {
            Self {
                a: starter,
                b: starter,
            }
        }

        pub fn double_a(self) -> Self {
            Self {
                a: self.a * 2,
                b: self.b,
            }
        }

        pub fn into_foo(self) -> Foo {
            Foo {
                inner: self.a + self.b,
            }
        }
    }
}

fn main() {
    let x = foo_module::FooBuilder::new(10)
        .double_a()
}
```

```

        .into_foo();

println!("{}", x);
}

```

не перекладено

не перекладено

5.1.1. не перекладено

не перекладено

не перекладено

5.2. не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

5.2.1. не перекладено

не перекладено

не перекладено	не перекладено	не перекладено	не перекладено	не перекладено
enable	0	0	disabled	не перекладено
		1	enabled	не перекладено
direction	1	0	input	не перекладено
		1	output	не перекладено
input_mode	2..3	00	hi-z	не перекладено
		01	pull-low	не перекладено
		10	pull-high	не перекладено
		11	n/a	не перекладено
output_mode	4	0	set-low	не перекладено
		1	set-high	не перекладено
input_status	5	x	in-val	не перекладено

не перекладено

```

/// не перекладено
struct GpioConfig {
    /// не перекладено
    periph: GPIO_CONFIG,
}

impl GpioConfig {
    pub fn set_enable(&mut self, is_enabled: bool) {
        self.periph.modify(|_r, w| {

```



```

        w.enable().set_bit(is_enabled)
    });
}

pub fn set_direction(&mut self, is_output: bool) {
    self.periph.modify(|_r, w| {
        w.direction().set_bit(is_output)
    });
}

pub fn set_input_mode(&mut self, variant: InputMode) {
    self.periph.modify(|_r, w| {
        w.input_mode().variant(variant)
    });
}

pub fn set_output_mode(&mut self, is_high: bool) {
    self.periph.modify(|_r, w| {
        w.output_mode.set_bit(is_high)
    });
}

pub fn get_input_status(&self) -> bool {
    self.periph.read().input_status().bit_is_set()
}
}

```

не перекладено

не перекладено

не перекладено

5.3. не перекладено

не перекладено

не перекладено	не перекладено	не перекладено	не перекладено	не перекладено
enable	0	0	disabled	не перекладено
		1	enabled	не перекладено
direction	1	0	input	не перекладено
		1	output	не перекладено
input_mode	2..3	00	hi-z	не перекладено
		01	pull-low	не перекладено
		10	pull-high	не перекладено
		11	n/a	не перекладено
output_mode	4	0	set-low	не перекладено
		1	set-high	не перекладено
input_status	5	x	in-val	не перекладено

не перекладено

```

/// не перекладено
struct GpioConfig {
    /// не перекладено
    periph: GPIO_CONFIG,
}

impl GpioConfig {
    pub fn set_enable(&mut self, is_enabled: bool) {
        self.periph.modify(|_r, w| {
            w.enable().set_bit(is_enabled)
        });
    }

    pub fn set_direction(&mut self, is_output: bool) -> Result<(), ()> {
        if self.periph.read().enable().bit_is_clear() {
            // не перекладено
            return Err(());
        }

        self.periph.modify(|r, w| {
            w.direction().set_bit(is_output)
        });

        Ok(())
    }

    pub fn set_input_mode(&mut self, variant: InputMode) -> Result<(), ()> {
        if self.periph.read().enable().bit_is_clear() {
            // не перекладено
            return Err(());
        }
    }
}

```

```

    }

    if self.periph.read().direction().bit_is_set() {
        // не перекладено
        return Err(());
    }

    self.periph.modify(|_r, w| {
        w.input_mode().variant(variant)
    });

    Ok(())
}

pub fn set_output_status(&mut self, is_high: bool) -> Result<(), ()> {
    if self.periph.read().enable().bit_is_clear() {
        // не перекладено
        return Err(());
    }

    if self.periph.read().direction().bit_is_clear() {
        // не перекладено
        return Err(());
    }

    self.periph.modify(|_r, w| {
        w.output_mode.set_bit(is_high)
    });

    Ok(())
}

pub fn get_input_status(&self) -> Result<bool, ()> {
    if self.periph.read().enable().bit_is_clear() {
        // не перекладено
        return Err(());
    }

    if self.periph.read().direction().bit_is_set() {
        // не перекладено
        return Err(());
    }

    Ok(self.periph.read().input_status().bit_is_set())
}
}

```

не перекладено

5.3.1. не перекладено

не перекладено

```

/// не перекладено
struct GpioConfig<ENABLED, DIRECTION, MODE> {
    /// не перекладено
    periph: GPIO_CONFIG,
    enabled: ENABLED,
    direction: DIRECTION,
    mode: MODE,
}

// не перекладено
struct Disabled;
struct Enabled;
struct Output;
struct Input;
struct PulledLow;
struct PulledHigh;
struct HighZ;
struct DontCare;

/// не перекладено
impl<EN, DIR, IN_MODE> GpioConfig<EN, DIR, IN_MODE> {
    pub fn into_disabled(self) -> GpioConfig<Disabled, DontCare, DontCare> {
        self.periph.modify(|_r, w| w.enable.disabled());
        GpioConfig {
            periph: self.periph,
            enabled: Disabled,
            direction: DontCare,
            mode: DontCare,
        }
    }

    pub fn into_enabled_input(self) -> GpioConfig<Enabled, Input, HighZ> {
        self.periph.modify(|_r, w| {
            w.enable.enabled()
                .direction.input()
                .input_mode.high_z()
        });
        GpioConfig {
            periph: self.periph,
            enabled: Enabled,
            direction: Input,
            mode: HighZ,
        }
    }
}

```

```

    }
}

pub fn into_enabled_output(self) -> GpioConfig<Enabled, Output, DontCare> {
    self.periph.modify(|_r, w| {
        w.enable.enabled()
            .direction.output()
            .input_mode.set_high()
    });
    GpioConfig {
        periph: self.periph,
        enabled: Enabled,
        direction: Output,
        mode: DontCare,
    }
}

/// не перекладено
impl GpioConfig<Enabled, Output, DontCare> {
    pub fn set_bit(&mut self, set_high: bool) {
        self.periph.modify(|_r, w| w.output_mode.set_bit(set_high));
    }
}

/// не перекладено
impl<IN_MODE> GpioConfig<Enabled, Input, IN_MODE> {
    pub fn bit_is_set(&self) -> bool {
        self.periph.read().input_status.bit_is_set()
    }

    pub fn into_input_high_z(self) -> GpioConfig<Enabled, Input, HighZ> {
        self.periph.modify(|_r, w| w.input_mode().high_z());
        GpioConfig {
            periph: self.periph,
            enabled: Enabled,
            direction: Input,
            mode: HighZ,
        }
    }

    pub fn into_input_pull_down(self) -> GpioConfig<Enabled, Input, PulledLow> {
        self.periph.modify(|_r, w| w.input_mode().pull_low());
        GpioConfig {
            periph: self.periph,
            enabled: Enabled,

```

```

        direction: Input,
        mode: PulledLow,
    }
}

pub fn into_input_pull_up(self) -> GpioConfig<Enabled, Input, PulledHigh> {
    self.periph.modify(|_r, w| w.input_mode().pull_high());
    GpioConfig {
        periph: self.periph,
        enabled: Enabled,
        direction: Input,
        mode: PulledHigh,
    }
}
}

```

не перекладено

```

/*
 * не перекладено
 */
let pin: GpioConfig<Disabled, _, _> = get_gpio();

// не перекладено
// pin.into_input_pull_down();

// не перекладено
let input_pin = pin.into_enabled_input();

// не перекладено
let pin_state = input_pin.bit_is_set();

// не перекладено
// input_pin.set_bit(true);

/*
 * не перекладено
 */
let pulled_low = input_pin.into_input_pull_down();
let pin_state = pulled_low.bit_is_set();

/*
 * не перекладено
 */
let output_pin = pulled_low.into_enabled_output();
output_pin.set_bit(true);

```



```
// не перекладено
// output_pin.into_input_pull_down();
```

не перекладено

5.3.2. не перекладено

не перекладено

не перекладено

5.4. не перекладено

не перекладено

```
use core::mem::size_of;

let _ = size_of::<Enabled>(); // == 0
let _ = size_of::<Input>(); // == 0
let _ = size_of::<PulledHigh>(); // == 0
let _ = size_of::<GpioConfig<Enabled, Input, PulledHigh>>(); // == 0
```

 Rust

5.4.1. не перекладено

```
struct Enabled;
```

 Rust

не перекладено

не перекладено

```
pub fn into_input_high_z(self) -> GpioConfig<Enabled, Input, HighZ> {
    self.periph.modify(|_r, w| w.input_mode().high_z());
    GpioConfig {
        periph: self.periph,
        enabled: Enabled,
        direction: Input,
        mode: HighZ,
    }
}
```

 Rust

не перекладено

5.4.2. не перекладено

не перекладено

не перекладено

6. не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

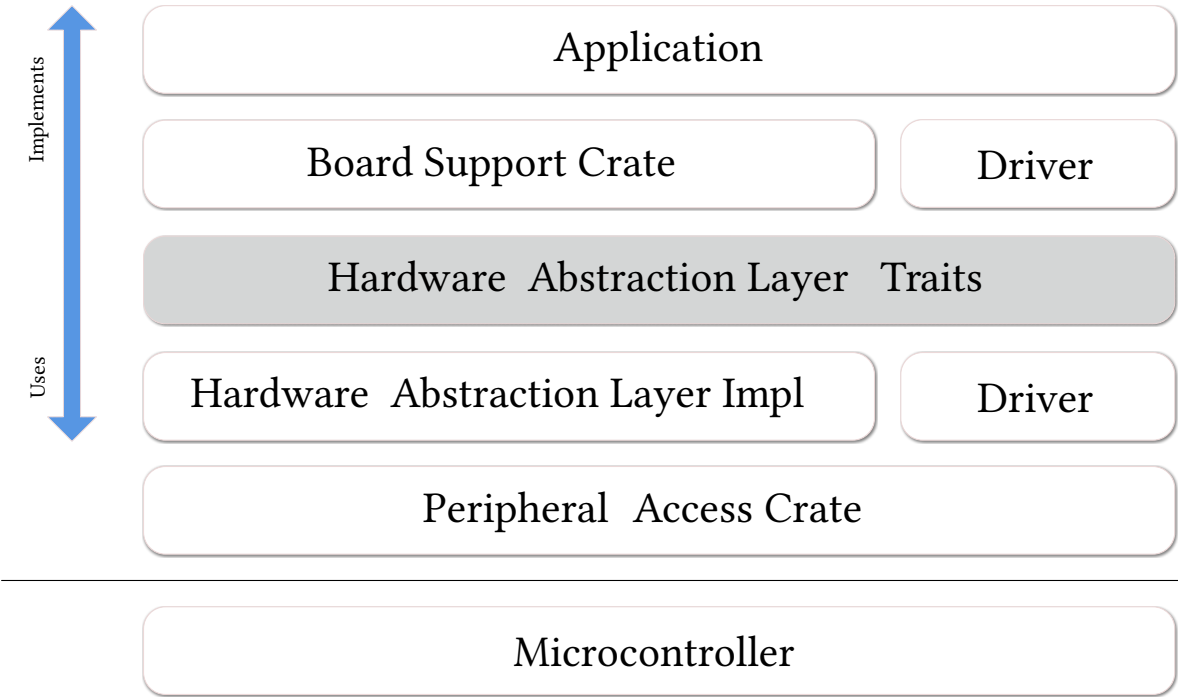
не перекладено

не перекладено

6.0.1. не перекладено

не перекладено

не перекладено



не перекладено

не перекладено

6.0.2. не перекладено

не перекладено

6.0.2.1. не перекладено

не перекладено

не перекладено

6.0.2.2. не перекладено

не перекладено

не перекладено

6.0.2.3. не перекладено
не перекладено

7. не перекладено

не перекладено

не перекладено

7.0.1. не перекладено

не перекладено

```
#[entry]
fn main() {
    let peripherals = setup_peripherals();
    loop {
        let inputs = read_inputs(&peripherals);
        let outputs = process(inputs);
        write_outputs(&peripherals, outputs);
    }
}
```

не перекладено

7.0.2. не перекладено

не перекладено

не перекладено

не перекладено

```
static mut COUNTER: u32 = 0;

#[entry]
fn main() -> ! {
    set_timer_1hz();
    let mut last_state = false;
    loop {
        let state = read_signal_level();
        if state && !last_state {
            // не перекладено
            unsafe { COUNTER += 1 };
        }
        last_state = state;
    }
}

#[interrupt]
fn timer() {
    unsafe { COUNTER = 0; }
}
```

не перекладено

7.0.3. не перекладено

не перекладено

```
static mut COUNTER: u32 = 0;

#[entry]
fn main() -> ! {
    set_timer_1hz();
    let mut last_state = false;
    loop {
        let state = read_signal_level();
        if state && !last_state {
            // не перекладено
            cortex_m::interrupt::free(|_| {
                unsafe { COUNTER += 1 };
            });
        }
        last_state = state;
    }
}

#[interrupt]
fn timer() {
    unsafe { COUNTER = 0; }
}
```

не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

7.0.4. не перекладено

не перекладено

```
use core::sync::atomic::{AtomicUsize, Ordering};

static COUNTER: AtomicUsize = AtomicUsize::new(0);

#[entry]
fn main() -> ! {
    set_timer_1hz();
    let mut last_state = false;
    loop {
        let state = read_signal_level();
        if state && !last_state {
```

```

        // не перекладено
        COUNTER.fetch_add(1, Ordering::Relaxed);
    }
    last_state = state;
}
}

#[interrupt]
fn timer() {
    // не перекладено
    COUNTER.store(0, Ordering::Relaxed)
}

```

не перекладено

не перекладено

не перекладено

7.0.5. не перекладено

не перекладено

не перекладено

```

use core::cell::UnsafeCell;
use cortex_m::interrupt;

// не перекладено
struct CSCounter(UnsafeCell<u32>);

const CS_COUNTER_INIT: CSCounter = CSCounter(UnsafeCell::new(0));

impl CSCounter {
    pub fn reset(&self, _cs: &interrupt::CriticalSection) {
        // не перекладено
        unsafe { *self.0.get() = 0 };
    }

    pub fn increment(&self, _cs: &interrupt::CriticalSection) {
        unsafe { *self.0.get() += 1 };
    }
}

// не перекладено
unsafe impl Sync for CSCounter {}

// не перекладено
static COUNTER: CSCounter = CS_COUNTER_INIT;

```

 Rust

```

#[entry]
fn main() -> ! {
    set_timer_1hz();
    let mut last_state = false;
    loop {
        let state = read_signal_level();
        if state && !last_state {
            // не перекладено
            interrupt::free(|cs| COUNTER.increment(cs));
        }
        last_state = state;
    }
}

#[interrupt]
fn timer() {
    // не перекладено
    interrupt::free(|cs| COUNTER.reset(cs));

    // не перекладено
    // let cs = unsafe { interrupt::CriticalSection::new() };
}

```

не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

7.0.6. не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

```

use core::cell::Cell;
use cortex_m::interrupt::Mutex;

static COUNTER: Mutex<Cell<u32>> = Mutex::new(Cell::new(0));

#[entry]
fn main() -> ! {
    set_timer_1hz();

```



```

let mut last_state = false;
loop {
    let state = read_signal_level();
    if state && !last_state {
        interrupt::free(|cs|
            COUNTER.borrow(cs).set(COUNTER.borrow(cs).get() + 1));
    }
    last_state = state;
}
}

#[interrupt]
fn timer() {
    // не перекладено
    interrupt::free(|cs| COUNTER.borrow(cs).set(0));
}

```

не перекладено

не перекладено

не перекладено

7.0.7. не перекладено

не перекладено

не перекладено

не перекладено

```

use core::cell::RefCell;
use cortex_m::interrupt::{self, Mutex};
use stm32f4::stm32f405;

static MY_GPIO: Mutex<RefCell<Option<stm32f405::GPIOA>>> =
    Mutex::new(RefCell::new(None));

#[entry]
fn main() -> ! {
    // не перекладено
    let dp = stm32f405::Peripherals::take().unwrap();
    let gpioa = &dp.GPIOA;

    // не перекладено
    configure_gpio(gpioa);

    // не перекладено
    interrupt::free(|cs| MY_GPIO.borrow(cs).replace(Some(dp.GPIOA)));
    // не перекладено
    set_timer_1hz();
}

```



```

let mut last_state = false;
loop {
    // не перекладено
    let state = interrupt::free(|cs| {
        let gpioa = MY_GPIO.borrow(cs).borrow();
        gpioa.as_ref().unwrap().idr.read().idr0().bit_is_set()
    });

    if state && !last_state {
        // не перекладено
        interrupt::free(|cs| {
            let gpioa = MY_GPIO.borrow(cs).borrow();
            gpioa.as_ref().unwrap().odr.modify(|_, w| w.odr1().set_bit());
        });
        last_state = state;
    }
}

#[interrupt]
fn timer() {
    // не перекладено
    interrupt::free(|cs| {
        // не перекладено
        let gpioa = MY_GPIO.borrow(cs).borrow();
        gpioa.as_ref().unwrap().odr.modify(|_, w| w.odr1().clear_bit());
    });
}

```

не перекладено

```

static MY_GPIO: Mutex<RefCell<Option<stm32f405::GPIOA>>> =
    Mutex::new(RefCell::new(None));

```

 Rust

не перекладено

```

interrupt::free(|cs| MY_GPIO.borrow(cs).replace(Some(dp.GPIOA)));

```

 Rust

не перекладено

```

interrupt::free(|cs| {
    let gpioa = MY_GPIO.borrow(cs).borrow();
    gpioa.as_ref().unwrap().odr.modify(|_, w| w.odr1().set_bit());
});

```

 Rust

не перекладено

не перекладено

не перекладено

```

use core::cell::RefCell;
use core::ops::DerefMut;
use cortex_m::interrupt::{self, Mutex};
use cortex_m::asm::wfi;
use stm32f4::stm32f405;

static G_TIM: Mutex<RefCell<Option<Timer<stm32::TIM2>>>> =
    Mutex::new(RefCell::new(None));

#[entry]
fn main() -> ! {
    let mut cp = cm::Peripherals::take().unwrap();
    let dp = stm32f405::Peripherals::take().unwrap();

    // не перекладено
    let tim = configure_timer_interrupt(&mut cp, dp);

    interrupt::free(|cs| {
        G_TIM.borrow(cs).replace(Some(tim));
    });

    loop {
        wfi();
    }
}

#[interrupt]
fn timer() {
    interrupt::free(|cs| {
        if let Some(ref mut tim) = G_TIM.borrow(cs).borrow_mut().deref_mut() {
            tim.start(1.hz());
        }
    });
}

```

не перекладено

7.0.8. RTIC

не перекладено

не перекладено

не перекладено

7.0.9. Embassy

не перекладено

не перекладено

не перекладено

7.0.10. не перекладено

не перекладено

не перекладено

7.0.11. не перекладено

не перекладено

не перекладено

не перекладено

8. не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

8.0.1. не перекладено

не перекладено

```
#![feature(alloc)]
```



```
extern crate alloc;
```

```
use alloc::vec::Vec;
```

не перекладено

не перекладено

```
// не перекладено
```



```
use core::alloc::{GlobalAlloc, Layout};
```

```
use core::cell::UnsafeCell;
```

```
use core::ptr;
```

```
use cortex_m::interrupt;
```

```
// не перекладено
```

```
struct BumpPointerAlloc {
    head: UnsafeCell<usize>,
    end: usize,
}
```

```
unsafe impl Sync for BumpPointerAlloc {}
```

```
unsafe impl GlobalAlloc for BumpPointerAlloc {
    unsafe fn alloc(&self, layout: Layout) -> *mut u8 {
        // не перекладено
        interrupt::free(|_| {
            let head = self.head.get();
            let size = layout.size();
            let align = layout.align();
            let align_mask = !(align - 1);

            // не перекладено
            let start = (*head + align - 1) & align_mask;
```

```

        if start + size > self.end {
            // не перекладено
            ptr::null_mut()
        } else {
            *head = start + size;
            start as *mut u8
        }
    })
}

unsafe fn dealloc(&self, _: *mut u8, _: Layout) {
    // не перекладено
}

// не перекладено
#[global_allocator]
static HEAP: BumpPointerAlloc = BumpPointerAlloc {
    head: UnsafeCell::new(0x2000_0100),
    end: 0x2000_0200,
};

```

не перекладено

```

#![feature(alloc_error_handler)]

use cortex_m::asm;

#[alloc_error_handler]
fn on_oom(_layout: Layout) -> ! {
    asm::bkpt();

    loop {}
}

```



не перекладено

```

#[entry]
fn main() -> ! {
    let mut xs = Vec::new();

    xs.push(42);
    assert!(xs.pop(), Some(42));

    loop {
        // ..
    }
}

```



```
}
```

не перекладено

8.0.2. не перекладено

не перекладено

```
// heapless version: v0.4.x
use heapless::Vec;
use heapless::consts::*;

#[entry]
fn main() -> ! {
    let mut xs: Vec<_, U8> = Vec::new();

    xs.push(42).unwrap();
    assert_eq!(xs.pop(), Some(42));
    loop {}
}
```



не перекладено

не перекладено

не перекладено

не перекладено

8.0.3. не перекладено

не перекладено

8.0.3.1. не перекладено

не перекладено

не перекладено

не перекладено

8.0.3.2. не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

8.0.3.3. не перекладено

не перекладено

не перекладено

не перекладено

8.0.3.4. не перекладено

не перекладено

не перекладено

9. не перекладено

не перекладено

9.1. не перекладено

не перекладено

9.1.1. не перекладено

не перекладено

9.1.2. не перекладено

9.1.2.1. не перекладено (C-CRATE-NAME)

не перекладено

9.1.3. не перекладено

9.1.3.1. не перекладено (C-FREE)

не перекладено

не перекладено

не перекладено

не перекладено

```
# pub struct TIMER0;
pub struct Timer(TIMER0);

impl Timer {
    pub fn new(periph: TIMER0) -> Self {
        Self(periph)
    }

    pub fn free(self) -> TIMER0 {
        self.0
    }
}
```

9.1.3.2. не перекладено (C-REEXPORT-PAC)

не перекладено

не перекладено

9.1.3.3. не перекладено (C-HAL-TRAITS)

не перекладено

не перекладено

9.1.4. не перекладено

9.1.4.1. не перекладено (C-CTOR)

не перекладено

не перекладено

9.1.4.2. не перекладено (C-INLINE)

не перекладено

9.1.5. не перекладено**9.1.5.1. не перекладено (C-ZST-PIN)**

не перекладено

не перекладено

не перекладено

```

pub struct PA0;
pub struct PA1;
// ...

pub struct PortA;

impl PortA {
    pub fn split(self) -> PortAPins {
        PortAPins {
            pa0: PA0,
            pa1: PA1,
            // ...
        }
    }
}

pub struct PortAPins {
    pub pa0: PA0,
    pub pa1: PA1,
    // ...
}

```

9.1.5.2. не перекладено (C-ERASED-PIN)

не перекладено

не перекладено

```

/// не перекладено
pub struct PA0;

impl PA0 {
    pub fn erase_pin(self) -> PA {
        PA { pin: 0 }
    }
}

/// не перекладено
pub struct PA {

```

```

    /// не перекладено
    pin: u8,
}

impl PA {
    pub fn erase_port(self) -> Pin {
        Pin {
            port: Port::A,
            pin: self.pin,
        }
    }
}

pub struct Pin {
    port: Port,
    pin: u8,
    /// не перекладено
}

enum Port {
    A,
    B,
    C,
    D,
}

```

9.1.5.3. не перекладено (C-PIN-STATE)

не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

- pub fn into_input<N: InputState>(self, input: N) -> Pin<N>
- pub fn into_output<N: OutputState>(self, output: N) -> Pin<N>

- ```
pub fn with_input_state<N: InputState, R>(
 &mut self,
 input: N,
 f: impl FnOnce(&mut PA1<N>) -> R,
) -> R
```


- ```
pub fn with_output_state<N: OutputState, R>(
    &mut self,
    output: N,
    f: impl FnOnce(&mut PA1<N>) -> R,
) -> R
```



не перекладено

не перекладено

```
# use std::marker::PhantomData;

mod sealed {
    pub trait Sealed {}
}

pub trait PinState: sealed::Sealed {}
pub trait OutputState: sealed::Sealed {}
pub trait InputState: sealed::Sealed {
    // ...
}

pub struct Output<S: OutputState> {
    _p: PhantomData<S>,
}

impl<S: OutputState> PinState for Output<S> {}
impl<S: OutputState> sealed::Sealed for Output<S> {}

pub struct PushPull;
pub struct OpenDrain;

impl OutputState for PushPull {}
impl OutputState for OpenDrain {}
impl sealed::Sealed for PushPull {}
impl sealed::Sealed for OpenDrain {}

pub struct Input<S: InputState> {
    _p: PhantomData<S>,
}

impl<S: InputState> PinState for Input<S> {}
impl<S: InputState> sealed::Sealed for Input<S> {}

pub struct Floating;
pub struct PullUp;
pub struct PullDown;

impl InputState for Floating {}
impl InputState for PullUp {}
impl InputState for PullDown {}
impl sealed::Sealed for Floating {}
impl sealed::Sealed for PullUp {}
impl sealed::Sealed for PullDown {}
```




```

pub struct PA1<S: PinState> {
    _p: PhantomData<S>,
}

impl<S: PinState> PA1<S> {
    pub fn into_input<N: InputState>(self, input: N) -> PA1<Input<N>> {
        todo!()
    }

    pub fn into_output<N: OutputState>(self, output: N) -> PA1<Output<N>> {
        todo!()
    }

    pub fn with_input_state<N: InputState, R>(
        &mut self,
        input: N,
        f: impl FnOnce(&mut PA1<N>) -> R,
    ) -> R {
        todo!()
    }

    pub fn with_output_state<N: OutputState, R>(
        &mut self,
        output: N,
        f: impl FnOnce(&mut PA1<N>) -> R,
    ) -> R {
        todo!()
    }
}

// не перекладено

```

10. не перекладено

не перекладено

10.0.1. не перекладено

не перекладено

не перекладено

10.0.1.1. не перекладено

не перекладено

не перекладено

```
[features]
```

 TOML

```
FIR = []
```

```
IIR = []
```

не перекладено

```
/// не перекладено
```

 Rust

```
#[cfg(feature="FIR")]
```

```
pub mod fir;
```

```
#[cfg(feature="IIR")]
```

```
pub mod iir;
```

не перекладено

не перекладено

не перекладено

10.0.1.2. не перекладено

не перекладено

```
const fn array_size() -> usize {
```

 Rust

```
    #[cfg(feature="use_more_ram")]
```

```
    { 1024 }
```

```
    #[cfg(not(feature="use_more_ram"))]
```

```
    { 128 }
```

```
}
```

```
static BUF: [u32; array_size()] = [0u32; array_size()];
```

не перекладено

10.0.1.3. не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

10.0.2. не перекладено

не перекладено

не перекладено

не перекладено

10.0.2.1. не перекладено

не перекладено

не перекладено

10.0.3. не перекладено

не перекладено

```
int16_t arr[16];
int i;
for(i=0; i<sizeof(arr)/sizeof(arr[0]); i++) {
    process(arr[i]);
}
```



не перекладено

не перекладено

```
let arr = [0u16; 16];
for element in arr.iter() {
    process(*element);
}
```



не перекладено

не перекладено

10.0.4. не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

10.0.5. не перекладено

не перекладено

не перекладено

не перекладено

```
volatile bool signalled = false;

void ISR() {
    // не перекладено
    signalled = true;
}
```



```

void driver() {
    while(true) {
        // не перекладено
        while(!signalled) { WFI(); }
        // не перекладено
        signalled = false;
        // не перекладено
        run_task();
    }
}

```

не перекладено

```

static mut SIGNALLED: bool = false;

#[interrupt]
fn ISR() {
    // не перекладено
    unsafe { core::ptr::write_volatile(&mut SIGNALLED, true) };
}

fn driver() {
    loop {
        // не перекладено
        while unsafe { !core::ptr::read_volatile(&SIGNALLED) } {}
        // не перекладено
        unsafe { core::ptr::write_volatile(&mut SIGNALLED, false) };
        // не перекладено
        run_task();
    }
}

```

 Rust

не перекладено

не перекладено

10.0.6. не перекладено

не перекладено

не перекладено

```

struct Foo {
    x: u16,
    y: u8,
    z: u16,
}

fn main() {

```

 Rust

```

let v = Foo { x: 0, y: 0, z: 0 };
println!("{:p} {:p} {:p}", &v.x, &v.y, &v.z);
}

// 0x7ffecb3511d0 0x7ffecb3511d4 0x7ffecb3511d2
// не перекладено

```

не перекладено

```

#[repr(C)]
struct Foo {
    x: u16,
    y: u8,
    z: u16,
}

fn main() {
    let v = Foo { x: 0, y: 0, z: 0 };
    println!("{:p} {:p} {:p}", &v.x, &v.y, &v.z);
}

// 0x7fffd0d84c60 0x7fffd0d84c62 0x7fffd0d84c64
// не перекладено

```



не перекладено

```

#[repr(packed)]
struct Foo {
    x: u16,
    y: u8,
    z: u16,
}

fn main() {
    let v = Foo { x: 0, y: 0, z: 0 };
    // не перекладено
    let px = std::ptr::addr_of!(v.x);
    let py = std::ptr::addr_of!(v.y);
    let pz = std::ptr::addr_of!(v.z);
    println!("{:p} {:p} {:p}", px, py, pz);
}

// 0x7ffd33598490 0x7ffd33598492 0x7ffd33598493
// не перекладено

```



не перекладено

не перекладено

```
#[repr(C)]
#[repr(align(4096))]
struct Foo {
    x: u16,
    y: u8,
    z: u16,
}

fn main() {
    let v = Foo { x: 0, y: 0, z: 0 };
    let u = Foo { x: 0, y: 0, z: 0 };
    println!("{:p} {:p} {:p}", &v.x, &v.y, &v.z);
    println!("{:p} {:p} {:p}", &u.x, &u.y, &u.z);
}

// 0x7ffec909a000 0x7ffec909a002 0x7ffec909a004
// 0x7ffec909b000 0x7ffec909b002 0x7ffec909b004
// не перекладено
```

не перекладено

не перекладено

10.0.7. не перекладено

не перекладено

11. не перекладено

не перекладено

не перекладено

не перекладено

не перекладено	не перекладено	не перекладено
String	CString	char *
&str	CStr	const char *
()	c_void	void
не перекладено		

, c_uint, unsigned int, tr((en: [etc], de: [usw.], ja: [他],)), [...], [...],))

не перекладено

```
fn foo(num: u32) {
    let c_num: c_uint = num;
    let r_num: u32 = c_num;
}
```

 Rust

11.0.1. не перекладено

не перекладено

не перекладено

11.0.2. не перекладено

не перекладено

не перекладено

не перекладено

11.1. не перекладено

не перекладено

не перекладено

11.1.1. не перекладено

не перекладено

не перекладено

11.1.1.1. не перекладено

не перекладено

```
/* File: cool.h */
typedef struct CoolStruct {
    int x;
    int y;
} CoolStruct;

void cool_function(int i, char c, CoolStruct* cs);
```

 C

не перекладено

```
/* File: cool_bindings.rs */
#[repr(C)]
pub struct CoolStruct {
    pub x: cty::c_int,
    pub y: cty::c_int,
}

extern "C" {
    pub fn cool_function(
        i: cty::c_int,
        c: cty::c_char,
        cs: *mut CoolStruct
    );
}
```

не перекладено

```
#[repr(C)]
pub struct CoolStruct { ... }
```

не перекладено

```
pub x: cty::c_int,
pub y: cty::c_int,
```

не перекладено

```
extern "C" { pub fn cool_function( ... ); }
```

не перекладено

```
i: cty::c_int,
c: cty::c_char,
cs: *mut CoolStruct
```

не перекладено

не перекладено

11.1.1.2. не перекладено

не перекладено

11.1.2. не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

11.1.2.1. не перекладено

не перекладено

не перекладено

11.1.2.2. не перекладено

не перекладено

не перекладено

11.1.2.3. не перекладено

не перекладено

не перекладено

```
fn main() {
    cc::Build::new()
        .file("src/foo.c")
        .compile("foo");
}
```

 Rust

не перекладено

11.2. не перекладено

не перекладено

не перекладено

11.2.1. не перекладено

не перекладено

не перекладено

```
[lib]
name = "your_crate"
crate-type = ["cdylib"]      # не перекладено
# crate-type = ["staticlib"] # не перекладено
```

 TOML
11.2.2. не перекладено

не перекладено

11.2.2.1. #[no_mangle]

не перекладено

11.2.2.2. extern "C"

не перекладено

не перекладено

не перекладено

```
#[no_mangle]
pub extern "C" fn rust_function() {
```

 Rust

```
}
```

не перекладено

11.2.3. не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

не перекладено

```
#[no_mangle]
pub extern "C" fn rust_function() {}
```



не перекладено

```
void rust_function();
```



не перекладено

не перекладено

не перекладено

```
#include "my-rust-project.h"
rust_function();
```



12. не перекладено

12.1. не перекладено

не перекладено

12.1.1. не перекладено

не перекладено

не перекладено

```
[profile.release]
```

[T]TOML

```
# не перекладено
```

```
debug = true
```

не перекладено

не перекладено

не перекладено

12.1.1.1. не перекладено

не перекладено

не перекладено

не перекладено

```
# Cargo.toml
```

[T]TOML

```
[package]
```

```
name = "app"
```

```
# ..
```

```
[profile.dev.package."*"] # +
```

```
opt-level = "z" # +
```

не перекладено

```
$ cargo size --bin app -- -A
```

[Plain Text]

```
app :
```

section	size	addr
.vector_table	1024	0x80000000
.text	9060	0x80004000
.rodata	1708	0x80027800
.data	0	0x20000000
.bss	4	0x20000000

не перекладено

```
$ cargo size --bin app -- -A
```

[Plain Text]

```
app :
```

section	size	addr
.vector_table	1024	0x80000000
.text	3490	0x80004000

.rodata	1100	0x80011c0
.data	0	0x20000000
.bss	4	0x20000000

не перекладено

```
# ..
# не перекладено
[profile.dev.package.cortex-m-rt] # +
opt-level = 0 # +

# не перекладено
[profile.dev.package."*"]
codegen-units = 1 # не перекладено
opt-level = "z"
```

не перекладено

12.1.2. не перекладено

не перекладено

не перекладено

не перекладено

12.1.3. не перекладено

не перекладено

не перекладено

```
[profile.release]
# не перекладено
opt-level = "s"
```

не перекладено

не перекладено

```
# .cargo/config.toml
# не перекладено
[target.'cfg(all(target_arch = "arm", target_os = "none"))']
rustflags = [
  # ..
  "-C", "inline-threshold=123", # +
]
```

не перекладено

не перекладено

12.2. не перекладено

не перекладено

//! не перекладено



```
fn main() {
    let float: f32 = 4.82832;
    let floored_float = float.floor();

    let sqrt_of_four = floored_float.sqrt();

    let sinus_of_four = floored_float.sin();

    let exponential_of_four = floored_float.exp();
    println!("Floored test float {} to {}", float, floored_float);
    println!("The square root of {} is {}", floored_float, sqrt_of_four);
    println!("The sinus of four is {}", sinus_of_four);
    println!(
        "The exponential of four to the base e is {}",
        exponential_of_four
    )
}
```

не перекладено

#![no_main]



#![no_std]

use panic_halt as _;

use cortex_m_rt::entry;

use cortex_m_semihosting::{debug, hprintln};

use libm::{exp, floorf, sin, sqrtf};

#[entry]

fn main() -> ! {

let float = 4.82832;

let floored_float = floorf(float);

let sqrt_of_four = sqrtf(floored_float);

let sinus_of_four = sin(floored_float.into());

let exponential_of_four = exp(floored_float.into());

hprintln!("Floored test float {} to {}", float, floored_float).unwrap();

hprintln!("The square root of {} is {}", floored_float, sqrt_of_four).unwrap();

hprintln!("The sinus of four is {}", sinus_of_four).unwrap();

hprintln!(

"The exponential of four to the base e is {}",

exponential_of_four

```
)  
.unwrap();  
// не перекладено  
// debug::exit(debug::EXIT_SUCCESS);  
  
loop {}  
}
```

не перекладено

- [CMSIS DSP library binding](#)
- [constgebra](#)
- [micromath](#)
- [microfft](#)
- [nalgebra](#)

A. не перекладено

не перекладено

BSP

не перекладено

FPU

не перекладено

HAL

не перекладено

I2C

не перекладено

PAC

не перекладено

SPI

не перекладено

SVD

не перекладено

UART

не перекладено

USART

не перекладено